W9 - PRACTICE

FIREBASE (REST API)

Learning objectives

- ✓ **Fetch data** from Firebase REST API and transform data with **DTO**
- ✓ Use **POST or PATCH or PUT** to update the Firebase data
- ✓ Handle a **JOIN** on 2 collections, using various approaches
- ✓ Handle a **CACHE** on REPOSITORIES

How to start?

- ✓ Copy the **START CODE** into your current repository and push it

  W9 - Start Code

- ✓ Each task of this practice shall be related to commit(s) including the **tasks ID.**

  W9 -01 – Fetch and display songs

How to submit?

- ✓ Once finished, submit on MS Team
  - Your GitHub repository URL
  - This document if needed

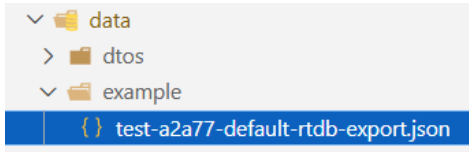
This practice focusses more **on clean code structure** rather than Flutter technical skills.
We will evaluate how well **you follow the coding conventions**, how your name your class, variables etc.

No code generated by AI is allowed

W9-01 – Fetch and display songs in the library screen

⚠️ Before starting this practice, you must have created your W9-DATABASE on Firebase with the right settings (rules…)

The start code is provided with a JSON file to be imported to your **firebase realtime database**



JSON data to import to your Firebase database

Refactor existing code

The JSON data does not exactly match with the current model/Song class.
As example, we don't store the artist's name but its ID.

```
},
"songs": {
  "song_1": {
    "title": "Time to Rise for Ronan",
    "duration": 210000,
    "artistId": "artist_1",
    "imageUrl": "https://images.unsplash.com/photo-1470225620780-dba8ba36b745"
  },
```

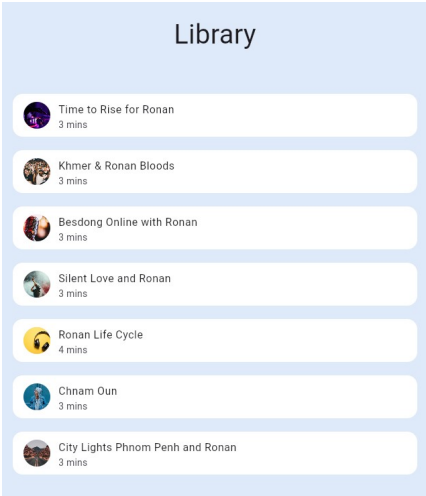
TODO

- ✓ Adapt the **Song class** in the /model accordingly
- ✓ Adapt the **SongDTO** accordingly
- ✓ Adapt the SongRepositoryFirebase

  Tip: The fetched data is not anymore, a List<dynamic> but a Map<String, dynamic>

- ✓ Adapt the **View** to display the image

  Tip: Use a CircleAvatar with a NetworkImage to display the image from URL



W9-02 – Fetch and display artists in the Artist screen

Follow the same approach to **fetch artists and display them** in a new view

TODO

- ✓ Implement the MODEL
- ✓ Implement the DTOS
- ✓ Implement the REPOSITORIES
- ✓ Inject the REPOSITORY to the main
- ✓ Create the Artists screen, model view and content
- ✓ Fetch the artists within the model view
- ✓ Display artist within an artist tile in the content

W9-03 – Join the Song with Artist to display rich Song information

We want now to display more information about the Artist on the Song Tile, as shown bellow:

❓ Where do you want to join the song and the artist collections? Repository? View Model? An additional Service?

❓ How to you want to store the Song additional information (artist name, artiste genre…)?

TODO

- ✓ Propose the updated architecture and sequence diagram – Validate it with the lecturer
- ✓ Implement your solution if validated